CES-AI Buddy-POC-Code Graph-Example

Development Reference · AI-Generated Technical Response

MODE

AppData+Feature

QUESTION

Track user login in CES database. Create new table if required. Is it possible to record user visited CES links and date and time time?

Auto-detected: view=CES\Views\Shared\EditorTemplates\Date.cshtml · controller=(none) · action=(none) · table=Users

DB

```
# DB Layer Analysis: User Login & Link Visit Tracking

## Current State
The `[SBCES].[Users]` table (schema.sql:2230-2243) stores user identity but has no
login/activity tracking columns. No existing table tracks user sessions, login timestamps, or
page/link visits.

## Required Changes

### 1. New Table: UserLoginHistory
Track each login event with timestamp.

### 2. New Table: UserActivityLog
Track each link/page visit with timestamp and URL.

Both tables use IDENTITY per the HAS_IDENTITY pattern shown in CONTEXT (Sites:2040,
Roles:2012, ErrorLog:1581, Users:2231).

---

## CHANGE Blocks

### CHANGE 1: DB - schema.sql
**OP:** NEW_FILE
**FILE:** Migrations\001_UserLoginTracking.sql

**CODE:**
```sql
-- Create UserLoginHistory table to track user login events
CREATE TABLE [SBCES].[UserLoginHistory](
■[LoginHistoryId] [int] IDENTITY(1,1) NOT NULL,
■[UserID] [int] NOT NULL,
■[LoginDateTime] [datetime] NOT NULL,
■[IPAddress] [varchar](45) NULL,
■[UserAgent] [varchar](500) NULL,
■CONSTRAINT [PK_UserLoginHistory] PRIMARY KEY CLUSTERED
■(
■■[LoginHistoryId] ASC
■)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS =
ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY],
■CONSTRAINT [FK_UserLoginHistory_Users] FOREIGN KEY([UserID])
■■REFERENCES [SBCES].[Users] ([UserID])
) ON [PRIMARY]
GO

-- Create UserActivityLog table to track visited links/pages
CREATE TABLE [SBCES].[UserActivityLog](
```
```

```
■[ActivityLogId] [int] IDENTITY(1,1) NOT NULL,
■[UserID] [int] NOT NULL,
■[VisitDateTime] [datetime] NOT NULL,
■[RequestUrl] [varchar](2000) NOT NULL,
■[HttpMethod] [varchar](10) NULL,
■[ControllerName] [varchar](100) NULL,
■[ActionName] [varchar](100) NULL,
■[IPAddress] [varchar](45) NULL,
■CONSTRAINT [PK_UserActivityLog] PRIMARY KEY CLUSTERED
■(
■■[ActivityLogId] ASC
■)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS =
ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY],
■CONSTRAINT [FK_UserActivityLog_Users] FOREIGN KEY([UserID])
■■REFERENCES [SBCES].[Users] ([UserID])
) ON [PRIMARY]
GO

-- Create indexes for common query patterns
CREATE NONCLUSTERED INDEX [IX_UserLoginHistory_UserID_LoginDateTime]
ON [SBCES].[UserLoginHistory] ([UserID], [LoginDateTime] DESC)
GO

CREATE NONCLUSTERED INDEX [IX_UserActivityLog_UserID_VisitDateTime]
ON [SBCES].[UserActivityLog] ([UserID], [VisitDateTime] DESC)
GO

CREATE NONCLUSTERED INDEX [IX_UserActivityLog_VisitDateTime]
ON [SBCES].[UserActivityLog] ([VisitDateTime] DESC)
GO
```

WHY: Creates two new tracking tables with IDENTITY PKs and explicit named FK constraints to Users table per rule 5, following ErrorLog pattern (schema.sql:1580-1590) for audit/logging tables.

CHANGE 2: DB - schema.sql
OP: NEW_FILE
FILE: Migrations\001_UserLoginTracking_Rollback.sql

CODE:
```sql
-- Rollback script for UserLoginHistory and UserActivityLog tables

-- Drop indexes first
DROP INDEX IF EXISTS [IX_UserActivityLog_VisitDateTime] ON [SBCES].[UserActivityLog]
GO

DROP INDEX IF EXISTS [IX_UserActivityLog_UserID_VisitDateTime] ON [SBCES].[UserActivityLog]
GO

DROP INDEX IF EXISTS [IX_UserLoginHistory_UserID_LoginDateTime] ON [SBCES].[UserLoginHistory]
GO

-- Drop tables (FK constraints drop automatically with tables)
DROP TABLE IF EXISTS [SBCES].[UserActivityLog]
GO

DROP TABLE IF EXISTS [SBCES].[UserLoginHistory]
GO
```

WHY: Provides rollback script to cleanly remove tracking tables and indexes if needed.

```
## Summary

**Created:**
- `[SBCES].[UserLoginHistory]`: Tracks each login event (UserID, LoginDateTime, IPAddress,
UserAgent)
- `[SBCES].[UserActivityLog]`: Tracks each page/link visit (UserID, VisitDateTime, RequestUrl,
HttpMethod, ControllerName, ActionName, IPAddress)

Both tables:
- Use IDENTITY for primary keys (per HAS_IDENTITY pattern in CONTEXT)
- Have explicit named FK constraints to `[SBCES].[Users]`
- Include indexes on UserID + DateTime for efficient querying
- Store datetime values for precise timestamp tracking

**ALSO NEEDS (Controller, answered in that layer's own section):** Logic to INSERT into
UserLoginHistory on successful authentication and INSERT into UserActivityLog on each request
(likely via action filter or middleware).

**ALSO NEEDS (View, answered in that layer's own section):** Admin UI to view/filter login
history and activity logs if reporting interface is required.
```

CONTROLLER

```
# CONTROLLER Layer Analysis: User Login & Link Visit Tracking

## Current State
BaseController.cs:51-116 shows `OnActionExecutionAsync` already runs on every request,
extracting UserADId from Windows auth (line 55-57), querying Users/Roles/Sites (lines 63-82),
and setting session/ViewBag values (lines 86-95). No INSERT to any tracking table occurs.

## Required Changes

### 1. Track Login Events
Insert into `[SBCES].[UserLoginHistory]` when a user's session is first established (UserADId
found in Users table but no existing session).

### 2. Track Page/Link Visits
Insert into `[SBCES].[UserActivityLog]` on every request in `OnActionExecutionAsync`.

Both use the controller's existing `_dbConnection` (Dapper) per rule 7.b - no new service
class.

---

## CHANGE Blocks

### CHANGE 1: Controller - Controllers\Base\BaseController.cs
**OP:** MODIFY
**ANCHOR:**
```csharp
        public override async Task OnActionExecutionAsync(ActionExecutingContext context,
ActionExecutionDelegate next)
        {
            try
            {
                string userADId = _httpContextAccessor.HttpContext.User.Identity.Name;
                string[] temp = userADId.Split('\\');
                userADId = temp[1];
               // var userADId__ =
_httpContextAccessor.HttpContext.User.FindFirst(ClaimTypes.NameIdentifier).Value;
                var userADId_ = Environment.UserName; // Get the username from the environment
                var userDomainName = Environment.UserDomainName;

                var userName =
```
```

```
                        _dbConnection.QueryFirstOrDefault<string>(@"SELECT u.UserName FROM
SBCES.Users u WHERE u.UserADId = '" + userADId + "'");

                if (string.IsNullOrEmpty(userName))
                {
                    context.Result= View("~/Views/Shared/Unauthorized.cshtml");
                    return;
                }
```
CODE:
```csharp
        public override async Task OnActionExecutionAsync(ActionExecutingContext context,
ActionExecutionDelegate next)
        {
            try
            {
                string userADId = _httpContextAccessor.HttpContext.User.Identity.Name;
                string[] temp = userADId.Split('\\');
                userADId = temp[1];
               // var userADId__ =
_httpContextAccessor.HttpContext.User.FindFirst(ClaimTypes.NameIdentifier).Value;
                var userADId_ = Environment.UserName; // Get the username from the environment
                var userDomainName = Environment.UserDomainName;

                var userName =
                        _dbConnection.QueryFirstOrDefault<string>(@"SELECT u.UserName FROM
SBCES.Users u WHERE u.UserADId = '" + userADId + "'");

                if (string.IsNullOrEmpty(userName))
                {
                    context.Result= View("~/Views/Shared/Unauthorized.cshtml");
                    return;
                }

                // Get UserId for tracking
                var userId = _dbConnection.QueryFirstOrDefault<int>(@"SELECT u.UserId FROM
SBCES.Users u WHERE u.UserADId = '" + userADId + "'");
```
WHY: Add UserId query needed for both tracking tables' UserID foreign key column.

CHANGE 2: Controller - Controllers\Base\BaseController.cs
OP: ADD_AFTER
ANCHOR:
```csharp
                ViewBag.SessionTimeoutMinutes = sessionTimeoutMinutes;
                ViewBag.SessionTimeoutWarningMinutes = sessionTimeoutWarningMinutes;

                await next();
```
CODE:
```csharp
                ViewBag.SessionTimeoutMinutes = sessionTimeoutMinutes;
                ViewBag.SessionTimeoutWarningMinutes = sessionTimeoutWarningMinutes;

                // Track login event if this is a new session
                await TrackLoginIfNewSession(userId, userADId);

                // Track page visit
                await TrackPageVisit(userId, context);

                await next();
```
WHY: Insert tracking calls before action execution to capture login and page visit events.

````
### CHANGE 3: Controller - Controllers\Base\BaseController.cs
**OP:** ADD_BEFORE
**ANCHOR:**
```csharp
        protected IActionResult HandleError(Exception ex)
        {
```
**CODE:**
```csharp
        private async Task TrackLoginIfNewSession(int userId, string userADId)
        {
            try
            {
                // Check if user has an active session (session was just created or doesn't
have UserADId yet)
                var existingSessionUser = HttpContext.Session.GetString("UserADId");

                // Only log login if this is a new session (no existing UserADId in session)
                if (string.IsNullOrEmpty(existingSessionUser))
                {
                    var ipAddress =
_httpContextAccessor.HttpContext.Connection.RemoteIpAddress?.ToString();
                    var userAgent =
_httpContextAccessor.HttpContext.Request.Headers["User-Agent"].ToString();

                    await _dbConnection.ExecuteAsync(
                        @"INSERT INTO [SBCES].[UserLoginHistory]
                          (UserID, LoginDateTime, IPAddress, UserAgent)
                          VALUES
                          (@UserID, @LoginDateTime, @IPAddress, @UserAgent)",
                        new
                        {
                            UserID = userId,
                            LoginDateTime = DateTime.Now,
                            IPAddress = ipAddress?.Length > 45 ? ipAddress.Substring(0, 45) :
ipAddress,
                            UserAgent = userAgent?.Length > 500 ? userAgent.Substring(0, 500)
: userAgent
                        });
                }
            }
            catch (Exception ex)
            {
                // Log error but don't block request
                await _dbConnection.ExecuteAsync(
                    @"INSERT INTO SBCES.ErrorLog
                      (UserName, ExceptionMessage, StackTrace)
                      VALUES
                      (@UserName, @ExceptionMessage, @StackTrace)",
                    new
                    {
                        UserName = "System_LoginTracking",
                        ExceptionMessage = ex.Message,
                        StackTrace = ex.ToString()
                    });
            }
        }

        private async Task TrackPageVisit(int userId, ActionExecutingContext context)
        {
            try
            {
                var request = _httpContextAccessor.HttpContext.Request;
                var ipAddress =
_httpContextAccessor.HttpContext.Connection.RemoteIpAddress?.ToString();
                var requestUrl =
$"{request.Scheme}://{request.Host}{request.Path}{request.QueryString}";
````
````

```
                var httpMethod = request.Method;

                // Extract controller and action names from route data
                var controllerName = context.RouteData.Values["controller"]?.ToString();
                var actionName = context.RouteData.Values["action"]?.ToString();

                await _dbConnection.ExecuteAsync(
                    @"INSERT INTO [SBCES].[UserActivityLog]
                      (UserID, VisitDateTime, RequestUrl, HttpMethod, ControllerName,
ActionName, IPAddress)
                      VALUES
                      (@UserID, @VisitDateTime, @RequestUrl, @HttpMethod, @ControllerName,
@ActionName, @IPAddress)",
                    new
                    {
                        UserID = userId,
                        VisitDateTime = DateTime.Now,
                        RequestUrl = requestUrl?.Length > 2000 ? requestUrl.Substring(0, 2000)
: requestUrl,
                        HttpMethod = httpMethod?.Length > 10 ? httpMethod.Substring(0, 10) :
httpMethod,
                        ControllerName = controllerName?.Length > 100 ?
controllerName.Substring(0, 100) : controllerName,
                        ActionName = actionName?.Length > 100 ? actionName.Substring(0, 100) :
actionName,
                        IPAddress = ipAddress?.Length > 45 ? ipAddress.Substring(0, 45) :
ipAddress
                    });
            }
            catch (Exception ex)
            {
                // Log error but don't block request
                await _dbConnection.ExecuteAsync(
                    @"INSERT INTO SBCES.ErrorLog
                      (UserName, ExceptionMessage, StackTrace)
                      VALUES
                      (@UserName, @ExceptionMessage, @StackTrace)",
                    new
                    {
                        UserName = "System_ActivityTracking",
                        ExceptionMessage = ex.Message,
                        StackTrace = ex.ToString()
                    });
            }
        }

        protected IActionResult HandleError(Exception ex)
        {
```

WHY: Add private helper methods on controller (per rule 7.b) to INSERT into UserLoginHistory (new session only) and UserActivityLog (every request), using existing _dbConnection pattern from HandleError (lines 124-134), with error handling that logs to ErrorLog without blocking the request.

Summary

Modified:
- `Controllers\Base\BaseController.cs`: Added UserId query, two private tracking methods (`TrackLoginIfNewSession`, `TrackPageVisit`), and calls to both in `OnActionExecutionAsync`.

Tracking Logic:
- **Login tracking**: Inserts into `[SBCES].[UserLoginHistory]` only when session is new (no existing UserADId in session), capturing UserID, LoginDateTime, IPAddress, UserAgent.
- **Page visit tracking**: Inserts into `[SBCES].[UserActivityLog]` on every request, capturing UserID, VisitDateTime, RequestUrl, HttpMethod, ControllerName, ActionName, IPAddress.

```
- Both use async Dapper (`ExecuteAsync`) with the controller's existing `_dbConnection` (no
new service class per rule 7.b).
- String truncation prevents DB constraint violations for varchar columns.
- Errors log to ErrorLog table without blocking the user's request.

**ALSO NEEDS (View, answered in that layer's own section):** Admin UI to query/display
UserLoginHistory and UserActivityLog data if reporting interface is required.
```

JS

```
# JS Layer Analysis: User Login & Link Visit Tracking

## Current State
No JavaScript files in CONTEXT. The prior layers implemented:
- **DB Layer:** Created `UserLoginHistory` and `UserActivityLog` tables
- **Controller Layer:** Added tracking logic to `BaseController.OnActionExecutionAsync`
(automatic INSERT on every request)
- **View Layer:** Created `Views\Admin\UserActivityReport.cshtml` with Kendo grids and filter
controls

## Required Changes

The View layer's `UserActivityReport.cshtml` references several JavaScript event handlers and
functions that need implementation:
- `onLoginUserFilterChange` (login history user filter)
- `onLoginDateFilterChange` (login history date filters)
- `refreshLoginGrid` (login history refresh button)
- `onActivityUserFilterChange` (activity log user filter)
- `onActivityDateFilterChange` (activity log date filters)
- `onActivityUrlFilterChange` (activity log URL filter)
- `refreshActivityGrid` (activity log refresh button)

These handlers must read filter values and refresh the corresponding Kendo grids with filtered
data.

---

## CHANGE Blocks

### CHANGE 1: JS - Scripts\Admin\userActivityReport.js
**OP:** NEW_FILE

**CODE:**
```javascript
// User Activity Report - Login History and Page Visit Activity filtering and grid refresh

function onLoginUserFilterChange() {
    refreshLoginGrid();
}

function onLoginDateFilterChange() {
    refreshLoginGrid();
}

function refreshLoginGrid() {
    var grid = $("#loginHistoryGrid").data("kendoGrid");
    if (!grid) return;

    var userId = $("#loginUserFilter").data("kendoDropDownList").value();
    var fromDate = $("#loginFromDate").data("kendoDatePicker").value();
    var toDate = $("#loginToDate").data("kendoDatePicker").value();

    var dataSource = grid.dataSource;
    dataSource.transport.options.read.data = {
```
```

```
        userId: userId || null,
        fromDate: fromDate ? kendo.toString(fromDate, "yyyy-MM-dd") : null,
        toDate: toDate ? kendo.toString(toDate, "yyyy-MM-dd") : null
    };

    dataSource.page(1);
}

function onActivityUserFilterChange() {
    refreshActivityGrid();
}

function onActivityDateFilterChange() {
    refreshActivityGrid();
}

function onActivityUrlFilterChange() {
    refreshActivityGrid();
}

function refreshActivityGrid() {
    var grid = $("#activityLogGrid").data("kendoGrid");
    if (!grid) return;

    var userId = $("#activityUserFilter").data("kendoDropDownList").value();
    var fromDate = $("#activityFromDate").data("kendoDatePicker").value();
    var toDate = $("#activityToDate").data("kendoDatePicker").value();
    var urlFilter = $("#activityUrlFilter").val();

    var dataSource = grid.dataSource;
    dataSource.transport.options.read.data = {
        userId: userId || null,
        fromDate: fromDate ? kendo.toString(fromDate, "yyyy-MM-dd") : null,
        toDate: toDate ? kendo.toString(toDate, "yyyy-MM-dd") : null,
        urlFilter: urlFilter || null
    };

    dataSource.page(1);
}
```

WHY: Implements all filter change handlers and grid refresh functions referenced by the View layer's UserActivityReport.cshtml, passing filter values as data parameters to the grid's read action.

Summary

Created:
- `Scripts\Admin\userActivityReport.js`: Standalone JavaScript file containing all event handlers for the User Activity Report view's filter controls and grid refresh logic.

Functions implemented:
- `onLoginUserFilterChange()`: Triggers login history grid refresh when user filter changes
- `onLoginDateFilterChange()`: Triggers login history grid refresh when date filters change
- `refreshLoginGrid()`: Reads all login history filters and refreshes grid with filtered data
- `onActivityUserFilterChange()`: Triggers activity log grid refresh when user filter changes
- `onActivityDateFilterChange()`: Triggers activity log grid refresh when date filters change
- `onActivityUrlFilterChange()`: Triggers activity log grid refresh when URL filter changes
- `refreshActivityGrid()`: Reads all activity log filters and refreshes grid with filtered data

All functions follow the Kendo Grid data source pattern: retrieve filter control values, build a data object, assign it to `dataSource.transport.options.read.data`, and call `dataSource.page(1)` to trigger a filtered read.

ALSO NEEDS (View, answered in that layer's own section): The `UserActivityReport.cshtml`

```
view must reference this script file (e.g., `<script
src="~/Scripts/Admin/userActivityReport.js"></script>` in a `@section Scripts` block).
```

VIEW

````
# VIEW Layer Analysis: User Login & Link Visit Tracking

## Current State
The shared layout (`Views\Shared\_Layout.cshtml:1-400`) contains no UI for viewing login
history or activity logs. The DB layer created `UserLoginHistory` and `UserActivityLog`
tables; the Controller layer added tracking logic to `BaseController.OnActionExecutionAsync`.
No admin reporting interface exists yet.

## Required Changes

This feature's tracking logic is fully implemented in the Controller layer (automatic INSERT
on every request). The VIEW layer needs an admin UI to query and display the tracked data.

---

## CHANGE Blocks

### CHANGE 1: View - Views\Admin\UserActivityReport.cshtml
**OP:** NEW_FILE

**CODE:**
```cshtml
@{
    ViewData["Title"] = "User Activity Report";
}

<div class="container-fluid mt-3">
    <h2>User Activity Report</h2>

    <div class="row mb-3">
        <div class="col-md-12">
            @(Html.Kendo().TabStrip()
                .Name("activityTabs")
                .Items(items =>
                {
                    items.Add().Text("Login History")
                        .Selected(true)
                        .Content(@<text>
                            <div class="p-3">
                                <div class="row mb-3">
                                    <div class="col-md-3">
                                        <label>User:</label>
                                        @(Html.Kendo().DropDownList()
                                            .Name("loginUserFilter")
                                            .OptionLabel("All Users")
                                            .DataTextField("UserName")
                                            .DataValueField("UserId")
                                            .DataSource(source => source
                                                .Read(read => read.Action("GetUsers",
"UserActivityReport"))
                                            )
                                            .Events(e => e.Change("onLoginUserFilterChange"))
                                        )
                                    </div>
                                    <div class="col-md-3">
                                        <label>From Date:</label>
                                        @(Html.Kendo().DatePicker()
                                            .Name("loginFromDate")
                                            .Value(DateTime.Now.AddDays(-30))
````
````

```
                                        .Events(e => e.Change("onLoginDateFilterChange"))
                                    )
                                </div>
                                <div class="col-md-3">
                                    <label>To Date:</label>
                                    @(Html.Kendo().DatePicker()
                                        .Name("loginToDate")
                                        .Value(DateTime.Now)
                                        .Events(e => e.Change("onLoginDateFilterChange"))
                                    )
                                </div>
                                <div class="col-md-3 d-flex align-items-end">
                                    <button class="k-button k-button-md k-rounded-md
k-button-solid k-button-solid-primary" onclick="refreshLoginGrid()">
                                        <span class="k-icon k-i-reload"></span> Refresh
                                    </button>
                                </div>
                            </div>

                            @(Html.Kendo().Grid<dynamic>()
                                .Name("loginHistoryGrid")
                                .Columns(columns =>
                                {
                                    columns.Bound("UserName").Title("User
Name").Width(150);
                                    columns.Bound("LoginDateTime").Title("Login
Date/Time").Width(180).Format("{0:yyyy-MM-dd HH:mm:ss}");
                                    columns.Bound("IPAddress").Title("IP
Address").Width(150);
                                    columns.Bound("UserAgent").Title("User
Agent").Width(300);
                                })
                                .Pageable(p => p.PageSizes(new[] { 10, 25, 50, 100 }))
                                .Sortable()
                                .Filterable()
                                .Scrollable()
                                .DataSource(dataSource => dataSource
                                    .Ajax()
                                    .PageSize(25)
                                    .Read(read => read.Action("GetLoginHistory",
"UserActivityReport"))
                                    .ServerOperation(true)
                                )
                                .ToolBar(toolbar =>
                                {
                                    toolbar.Excel();
                                })
                                .Excel(excel => excel
                                    .FileName("LoginHistory_" +
DateTime.Now.ToString("yyyyMMdd_HHmmss") + ".xlsx")
                                    .Filterable(true)
                                    .ProxyURL(Url.Action("Excel_Export_Save",
"UserActivityReport"))
                                )
                            )
                        </div>
                    </text>);

                items.Add().Text("Page Visit Activity")
                    .Content(@<text>
                        <div class="p-3">
                            <div class="row mb-3">
                                <div class="col-md-2">
                                    <label>User:</label>
                                    @(Html.Kendo().DropDownList()
                                        .Name("activityUserFilter")
                                        .OptionLabel("All Users")
```

```
                                        .DataTextField("UserName")
                                        .DataValueField("UserId")
                                        .DataSource(source => source
                                            .Read(read => read.Action("GetUsers",
"UserActivityReport"))
                                        )
                                        .Events(e =>
e.Change("onActivityUserFilterChange"))
                                    )
                                </div>
                                <div class="col-md-2">
                                    <label>From Date:</label>
                                    @(Html.Kendo().DatePicker()
                                        .Name("activityFromDate")
                                        .Value(DateTime.Now.AddDays(-7))
                                        .Events(e =>
e.Change("onActivityDateFilterChange"))
                                    )
                                </div>
                                <div class="col-md-2">
                                    <label>To Date:</label>
                                    @(Html.Kendo().DatePicker()
                                        .Name("activityToDate")
                                        .Value(DateTime.Now)
                                        .Events(e =>
e.Change("onActivityDateFilterChange"))
                                    )
                                </div>
                                <div class="col-md-2">
                                    <label>Controller:</label>
                                    @(Html.Kendo().DropDownList()
                                        .Name("controllerFilter")
                                        .OptionLabel("All Controllers")
                                        .DataTextField("ControllerName")
                                        .DataValueField("ControllerName")
                                        .DataSource(source => source
                                            .Read(read => read.Action("GetControllers",
"UserActivityReport"))
                                        )
                                        .Events(e => e.Change("onControllerFilterChange"))
                                    )
                                </div>
                                <div class="col-md-2">
                                    <label>Action:</label>
                                    @(Html.Kendo().DropDownList()
                                        .Name("actionFilter")
                                        .OptionLabel("All Actions")
                                        .DataTextField("ActionName")
                                        .DataValueField("ActionName")
                                        .DataSource(source => source
                                            .Read(read => read.Action("GetActions",
"UserActivityReport"))
                                        )
                                        .Events(e => e.Change("onActionFilterChange"))
                                    )
                                </div>
                                <div class="col-md-2 d-flex align-items-end">
                                    <button class="k-button k-button-md k-rounded-md
k-button-solid k-button-solid-primary" onclick="refreshActivityGrid()">
                                        <span class="k-icon k-i-reload"></span> Refresh
                                    </button>
                                </div>
                            </div>

                            @(Html.Kendo().Grid<dynamic>()
                                .Name("activityLogGrid")
                                .Columns(columns =>
```

```
                                    {
                                        columns.Bound("UserName").Title("User
Name").Width(120);
                                        columns.Bound("VisitDateTime").Title("Visit
Date/Time").Width(160).Format("{0:yyyy-MM-dd HH:mm:ss}");

columns.Bound("ControllerName").Title("Controller").Width(120);
                                        columns.Bound("ActionName").Title("Action").Width(120);
                                        columns.Bound("HttpMethod").Title("Method").Width(80);
                                        columns.Bound("RequestUrl").Title("URL").Width(300);
                                        columns.Bound("IPAddress").Title("IP
Address").Width(120);
                                    })
                                    .Pageable(p => p.PageSizes(new[] { 10, 25, 50, 100 }))
                                    .Sortable()
                                    .Filterable()
                                    .Scrollable()
                                    .DataSource(dataSource => dataSource
                                        .Ajax()
                                        .PageSize(25)
                                        .Read(read => read.Action("GetActivityLog",
"UserActivityReport"))
                                        .ServerOperation(true)
                                    )
                                    .ToolBar(toolbar =>
                                    {
                                        toolbar.Excel();
                                    })
                                    .Excel(excel => excel
                                        .FileName("ActivityLog_" +
DateTime.Now.ToString("yyyyMMdd_HHmmss") + ".xlsx")
                                        .Filterable(true)
                                        .ProxyURL(Url.Action("Excel_Export_Save",
"UserActivityReport"))
                                    )
                                )
                            </div>
                        </text>);
                })
            )
        </div>
    </div>
</div>

<script>
    function onLoginUserFilterChange() {
        refreshLoginGrid();
    }

    function onLoginDateFilterChange() {
        refreshLoginGrid();
    }

    function refreshLoginGrid() {
        var grid = $("#loginHistoryGrid").data("kendoGrid");
        var userId = $("#loginUserFilter").data("kendoDropDownList").value();
        var fromDate = $("#loginFromDate").data("kendoDatePicker").value();
        var toDate = $("#loginToDate").data("kendoDatePicker").value();

        grid.dataSource.read({
            userId: userId,
            fromDate: fromDate ? kendo.toString(fromDate, "yyyy-MM-dd") : null,
            toDate: toDate ? kendo.toString(toDate, "yyyy-MM-dd") : null
        });
    }

    function onActivityUserFilterChange() {
```

```
        refreshActivityGrid();
    }

    function onActivityDateFilterChange() {
        refreshActivityGrid();
    }

    function onControllerFilterChange() {
        refreshActivityGrid();
    }

    function onActionFilterChange() {
        refreshActivityGrid();
    }

    function refreshActivityGrid() {
        var grid = $("#activityLogGrid").data("kendoGrid");
        var userId = $("#activityUserFilter").data("kendoDropDownList").value();
        var fromDate = $("#activityFromDate").data("kendoDatePicker").value();
        var toDate = $("#activityToDate").data("kendoDatePicker").value();
        var controller = $("#controllerFilter").data("kendoDropDownList").value();
        var action = $("#actionFilter").data("kendoDropDownList").value();

        grid.dataSource.read({
            userId: userId,
            fromDate: fromDate ? kendo.toString(fromDate, "yyyy-MM-dd") : null,
            toDate: toDate ? kendo.toString(toDate, "yyyy-MM-dd") : null,
            controllerName: controller,
            actionName: action
        });
    }

    $(document).ready(function () {
        // Initial load
        refreshLoginGrid();
        refreshActivityGrid();
    });
</script>
```

WHY: Creates admin UI with two tabs (Login History and Page Visit Activity) using Telerik TabStrip, Grid, DropDownList, and DatePicker controls per CONTEXT conventions, with filter controls and Excel export matching the inline editing pattern shown in _Layout.cshtml:98-241.

CHANGE 2: View - Views\Shared_Layout.cshtml
OP: ADD_AFTER

ANCHOR:
```csharp
                                    children.Add().Text("Announcement Settings")
                        @* .Action("Index", "Common", new { area = ""
}).ImageUrl(Url.Content("~/images/part.png")); *@
                        .Action("Index", "Content", new { area = ""
}).ImageUrl(Url.Content("~/images/part.png"));
                                });
```

CODE:
```csharp
                                    children.Add().Text("Announcement Settings")
                        @* .Action("Index", "Common", new { area = ""
}).ImageUrl(Url.Content("~/images/part.png")); *@
                        .Action("Index", "Content", new { area = ""
}).ImageUrl(Url.Content("~/images/part.png"));
```

```
                                    children.Add().Text("User Activity Report")
                        .Action("Index", "UserActivityReport", new { area = ""
}).ImageUrl(Url.Content("~/images/part.png"));
                                });
```

WHY: Adds "User Activity Report" menu item under Administration menu (only visible to RoleId 1/2/3 per existing if block at line 164) to navigate to the new reporting view.

Summary

Created:
- `Views\Admin\UserActivityReport.cshtml`: Two-tab admin UI for viewing login history and page visit activity with filters (user, date range, controller, action) and Excel export.

Modified:
- `Views\Shared\_Layout.cshtml`: Added "User Activity Report" menu item under Administration.

Kendo/Telerik Controls Used:
- `Html.Kendo().TabStrip()` (UserActivityReport.cshtml) - two-tab layout
- `Html.Kendo().Grid<dynamic>()` (UserActivityReport.cshtml) - loginHistoryGrid, activityLogGrid
- `Html.Kendo().DropDownList()` (UserActivityReport.cshtml) - user/controller/action filters
- `Html.Kendo().DatePicker()` (UserActivityReport.cshtml) - date range filters
- `Html.Kendo().Menu()` (_Layout.cshtml:101) - existing menu extended with new item

Shared/EditorTemplates/Partials Used:
- None (Date.cshtml shown in CONTEXT is not referenced by this view; DatePicker is instantiated directly via `Html.Kendo().DatePicker()`)

USED_BY:
- `Views\Shared\_Layout.cshtml` is the site-wide shared layout referenced by all pages (CONTEXT shows it contains the main menu and session timeout logic); adding a menu item here makes the report accessible from every page.

Regression Risk:
- Low: New menu item is additive only, placed inside existing RoleId 1/2/3 check (line 164-219 in CONTEXT), no changes to existing menu items or layout structure.

ALSO NEEDS (Controller, answered in that layer's own section):
`UserActivityReportController` with actions `Index`, `GetUsers`, `GetLoginHistory`, `GetActivityLog`, `GetControllers`, `GetActions`, `Excel_Export_Save` to serve the view and query the `UserLoginHistory` and `UserActivityLog` tables created by the DB layer.

Source validation: 4/4 code citations verified against the real files.

Generated October 28, 2026 at 02:32 PM